jackjackbits and **claude** docs: correct inaccurate privacy and metadata claims (#1485)

229a415 · 3 months ago

150 lines (91 loc) · 14.4 KB

Preview Code Blame Raw

bitchat Protocol Whitepaper

Version 2.0

Date: July 6, 2026

Abstract

bitchat is a decentralized, peer-to-peer messaging application for secure, private, censorship-resistant communication that works with or without the internet. Nearby devices form an ad-hoc Bluetooth Low Energy (BLE) mesh; distant peers are reached over the Nostr protocol when a connection exists. A layered store-and-forward stack — a persistent sender outbox, opportunistic couriers with a spray-and-wait copy budget, gossip-synced public history, and Nostr relay mailboxes — delivers messages to peers who are out of range at send time. This document describes the protocol and its delivery guarantees as implemented.

1. Design Goals

- **Confidentiality:** all private communication is end-to-end encrypted; intermediate nodes and couriers carry only opaque ciphertext.

- **Authentication:** peers are identified by cryptographic keys; announcements are signed and verified.
- **Resilience:** the network functions in lossy, low-bandwidth, partitioned environments with churning membership.
- **Eventual delivery:** a message to an out-of-range peer should still arrive — relayed by the mesh, carried by a moving person, or resting on an internet relay — within a bounded retention window.
- **Ephemerality by default:** conversation timelines live in memory only. Everything the store-and-forward stack persists is either sealed ciphertext or already-public broadcast traffic, and all of it dies with the panic wipe. Media is the exception: accepted images and voice notes are written to disk unsealed, protected by the platform's data-protection class rather than by app-layer encryption, and bounded by a storage quota.

2. Architecture Overview

Two transports implement a common `Transport` interface and are coordinated by a `MessageRouter`:

- **BLE mesh** — every device is simultaneously a GATT central and peripheral, relaying packets in a controlled flood. No infrastructure, pairing, or accounts.
- **Nostr** — private messages to mutual favorites travel in BitChat's app-specific encrypted envelopes over public relays (over Tor where enabled), bridging separate meshes through the internet.

The router prefers a live mesh link, falls back to Nostr, and engages the courier system when neither can deliver promptly.

3. Identity

Each device holds two long-term key pairs in the Keychain:

- a **Curve25519 static key** for Noise key agreement — its SHA-256 fingerprint is the peer's stable identity, and
- an **Ed25519 signing key** for packet signatures.

On the mesh, peers appear under a short 8-byte peer ID. That ID is **not ephemeral**: it is the first 8 bytes of the SHA-256 fingerprint of the device's Noise static key, so it is stable across sessions, reboots, and reinstalls that preserve the keychain, and it changes only when the identity itself is replaced by a panic wipe. Favoriting pins the full Noise public key so identity survives across sessions. Mutual favorites also exchange Nostr public keys for the internet path. Optional QR verification binds a nickname to a fingerprint in person.

Signed announcements additionally carry the nickname, the Noise static public key, and the Ed25519 signing public key in cleartext (§4.5), so a passive receiver in radio range can link a device across time and place regardless of the peer ID. Unlinkable presence is not a property this protocol currently provides; see §9.

4. BLE Mesh Layer

4.1 Packet Format

A compact binary header (version, type, TTL, timestamp, flags) is followed by an 8-byte sender ID, an optional 8-byte recipient ID, the payload, and an optional Ed25519 signature. Version 2 packets may carry an explicit source route. Signatures exclude the TTL byte so relays can decrement it without invalidating them.

Only `noiseEncrypted` and `noiseHandshake` packets are padded, toward 256/512/1024/2048-byte buckets; every other type — public messages, announcements, board posts, group messages, fragments, files, and voice frames — goes out at its natural length. Padding is PKCS#7-style with pad bytes equal to the pad length, and because that length must fit one byte, a frame needing more than 255 bytes to reach its bucket is emitted unpadded. Payload length is therefore observable for most traffic.

4.2 Flood Control

Relaying is a deterministic controlled flood tuned by local connection degree:

- **TTL:** packets originate with TTL 7. Relays clamp: dense graphs (≥ 6 links) cap broadcast TTL at 5; thin chains (≤ 2 links) relay at full incoming depth.
- **Deduplication:** an LRU seen-set (1000 entries, 5-minute expiry) keyed by sender, timestamp, type, and a payload digest drops duplicates. A scheduled relay is cancelled when a duplicate arrives first from another relay.
- **Jitter:** relays wait a random 10–220 ms (wider when dense) so duplicate suppression wins often.
- **Fanout subsetting:** broadcast messages are re-sent to a deterministic, message-ID-seeded subset of links (~$\log_2$ of degree) rather than all of them; announces, fragments,

and sync packets use full fanout. The ingress link is always excluded (split horizon).
- **Directed traffic** (handshakes, private messages, courier envelopes) relays deterministically with TTL − 1 and tight jitter, and is never subset.

4.3 Routing

Announcements carry up to 10 direct-neighbor IDs, giving each node a shallow topology map (60 s freshness). When a bidirectionally-confirmed path exists, packets are source-routed along it; otherwise — and whenever a route fails — delivery falls back to flooding.

4.4 Fragmentation

Packets exceeding the link MTU split into ~469-byte fragments (8-byte fragment ID, index/total header) that relay independently and reassemble at each receiving node (128 concurrent assemblies, 30 s timeout, 1 MiB cap).

4.5 Presence

Signed announcements propagate multi-hop: every 4 s while isolated, backing off to ~15–30 s (jittered) when connected. A verified announce retains a peer as *reachable* for 60 s after last contact. Connection scheduling is RSSI-gated with duty-cycled scanning to bound battery drain.

5. Encryption

5.1 Live Sessions: Noise XX

Connected peers establish sessions with the Noise `XX` pattern (Curve25519 / ChaCha20-Poly1305 / SHA-256), providing mutual authentication and forward secrecy. All private payloads — messages, delivery acks, read receipts — ride inside the session as typed ciphertext. Intermediate relays see only opaque `noiseEncrypted` packets.

5.2 Offline Seals: Noise X

Courier envelopes are sealed to the recipient's *static* key with the one-way Noise `X` pattern; the sender's identity is authenticated inside the ciphertext. **This path has no forward secrecy** — compromise of the recipient's static key exposes sealed-but-undelivered mail. A prekey scheme is future work.

5.3 Nostr Path

Private messages to mutual favorites use BitChat's proprietary private-envelope protocol. An unsigned inner message (kind 14) is encrypted and placed in a sender-signed seal (kind 13); that seal is encrypted again inside a public envelope (kind 1059) signed by a one-time key, so relays learn neither the stable sender identity nor the content. Each encrypted content field is `v2:` followed by base64url of a 24-byte nonce, XChaCha20-Poly1305 ciphertext, and its 16-byte tag. Keys come from secp256k1 ECDH and HKDF-SHA256 (the derivation reuses a "nip44-v2" info label but is not the NIP-44 key schedule).

This format reuses NIP-17/NIP-59 kind numbers but is **not NIP-17, NIP-44, or NIP-59 compatible** and interoperates only with BitChat clients. The outer `p` tag exposes the recipient's Nostr public key to relays; the plaintext and stable sender identity remain inside authenticated ciphertext. Public seal and envelope timestamps are randomized by up to ±15 minutes, while the actual message timestamp is encrypted. The protocol does not provide forward secrecy: compromise of the recipient's static Nostr private key can expose stored envelopes addressed to that key.

6. Store and Forward

Four mechanisms cover the "recipient is not here right now" problem. All persisted state is wiped by panic mode.

6.1 Sender Outbox

Private messages without a prompt route are retained per peer (100 messages/peer, 24 h TTL) and re-sent on reconnect events until a delivery or read ack clears them, or a resend cap (8 attempts) drops them with visible failure. The outbox persists to disk sealed under a ChaChaPoly key held only in the Keychain, so queued mail survives an app kill without ever storing plaintext.

6.2 Couriers

When no transport can deliver promptly, the message is sealed (§5.2) into a **courier envelope** and handed to up to 3 connected peers who may physically encounter the recipient:

- **Opaque addressing.** The only routing information is a 16-byte rotating recipient tag — an HMAC of the recipient's static key and the UTC day — computable solely by parties who already know that key. Couriers learn neither sender, recipient, nor content, and tags do not correlate across days.
- **Trust tiers.** Mutual favorites may deposit 5 envelopes each; any peer with a signature-verified announce may deposit 2, into a bounded pool (20 of 40 slots) that can never crowd

out favorites' mail. Envelopes are capped at 16 KiB and 24 h; overflow evicts oldest verified-tier mail first.

- **Deposit retry.** Queued messages are re-deposited whenever a new eligible courier connects, until 3 distinct couriers carry the message or it expires.
- **Spray and wait.** Envelopes carry a copy budget (initially 4, capped at 8). A courier meeting another eligible courier hands over half its remaining budget, so mail diffuses through a moving crowd instead of riding one person. Budgets, spray history, and carried mail persist across app restarts (iOS file protection).
- **Handover.** On a verified *direct* announce from the recipient, matching envelopes are delivered over the live link and removed. On a verified *relayed* announce, a copy floods toward the recipient as a directed packet while the carried original stays put, throttled to one attempt per envelope per 10 minutes.
- Receivers dedup by message ID, so redundant copies and the retained outbox original are harmless. Couriered mail from blocked senders is dropped at decryption time.

6.3 Public History (Gossip Sync)

Public broadcast messages are cached (1000 packets) and reconciled between peers every ~15 s using compact GCS filters: each side advertises what it holds, the other returns what is missing. Messages stay sync-able for **6 hours** and the cache persists to disk, so a device that walks between two partitions — or relaunches later — serves the room's recent history to whoever missed it. Fragments and file transfers keep a short 15-minute window.

6.4 Nostr Mailboxes

BitChat private envelopes rest on Nostr relays; clients re-subscribe with a 24-hour lookback on reconnect, covering the both-devices-offline case for mutual favorites whenever either side touches the internet.

6.5 Delivery Metrics

Bare local counters (deposits, handovers, sprays, opens, outbox flushes and drops — no identities, message IDs, or timestamps) let delivery behavior be measured on-device. They never leave the device and are cleared by the panic wipe.

7. Application Layer

- **Public chat** — signed broadcast messages within the mesh, backed by the gossip-synced history above.

- **Private chat** — end-to-end encrypted messages with delivery and read receipts, over mesh, courier, or Nostr.
- **Location channels** — geohash-scoped public rooms carried over Nostr relays for regional chat beyond radio range.
- **Favorites** — the mutual-trust relationship that unlocks Nostr delivery and the larger courier quota.
- **Media** — files and images fragment over the mesh (1 MiB cap, explicit accept before anything touches disk); couriers carry text only.
- **Panic wipe** — clears identity keys, favorites, carried courier mail, the sealed outbox, archived public history, and metrics.

8. Security Considerations

- **Relay nodes** cannot read private traffic; they forward opaque ciphertext. Padding applies to Noise frames only (§4.1), so other packet types relay at their natural length.
- **Couriers** are quota-bounded mailbags. A malicious courier can drop mail (redundant copies and deposit retry mitigate this) but cannot read it, link it across days, or amplify it — copy budgets are capped and every envelope is validated against size and lifetime policy on deposit.
- **Flooding abuse** is bounded by TTL clamps, deduplication, per-depositor quotas, connect-rate limits, and announce-rate limiting.
- **Replay** of public broadcasts is bounded by the 6-hour acceptance window plus deduplication; private payloads are protected by Noise nonces.
- **Metadata is the weakest part of this design, and the peer ID does not help.** The 8-byte sender ID in every packet header is derived from a never-rotating key (§3), and announcements publish the static keys and nickname in cleartext, so a passive listener can enumerate participants and follow a device between places. Announcements also carry up to ten direct-neighbor IDs (§4.3), which hands a single sniffer the local adjacency graph. Origin packets leave at the default TTL, so hop distance identifies the originator. Daily-rotating courier tags do limit correlation of carried mail, and Nostr traffic can ride Tor. Addressing the radio-layer exposure is future work (§9).
- **No forward secrecy for sealed mail or Nostr private envelopes** (§5.2–5.3) means compromise of a recipient's static key can expose retained ciphertext addressed to that key.

9. Future Work

- Prekey-based forward secrecy for courier envelopes.

- Couriered media beyond the 16 KiB text cap.
- Probabilistic relay and edge-of-network TTL boosting for very dense and very sparse graphs.
- Multi-hop courier routing informed by encounter history.
- **Rotating on-air identity.** Epoch-rotating peer IDs, with static-key disclosure moved inside the encrypted handshake and mutual favorites recognising each other through a tag derived from their shared secret, so presence stops being linkable across sessions (§3, §8).
- **Padding for non-Noise packet types**, and closing the gap where a frame needing more than 255 bytes of padding is emitted unpadded (§4.1).
- Making the neighbor list in announcements optional, or restricted to authenticated links (§4.3).

This document describes the protocol as implemented in the current release. The implementation is free and unencumbered software released into the public domain.